



Proposed Methodology: Robust SOH Estimation Framework using AE-BPNN and Transfer Learning

台灣科技大學 AI所

指導教授：羅一峰 博士

學生：林志龍 (M11452803)



Outline

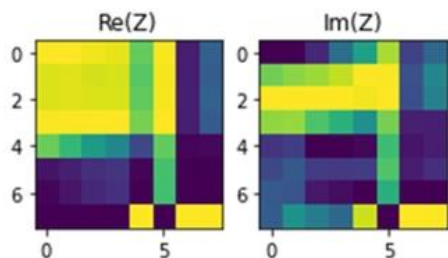
- ❑ Identifying the Gaps
- ❑ Model Selection: AE-BPNN & Denoising Concept
- ❑ Proposed Experiments (3 Scenarios)
- ❑ Open-Source Data Strategy
- ❑ Tentative Project Roadmap



Identifying the Gaps

Information Loss:

- **Issue:** 60 frequency points compressed into an 8×8 matrix.
- **Impact:** Significant loss of spatial correlation and dimensional details.



Reshape the data to be suitable for input of a 2D convolutional layer.
 X shape: $(n, 8, 8, 2)$

Data Scarcity:

- **Issue:** Limited laboratory dataset (Only 8 coin cells).
- **Impact:** High risk of model overfitting and limited training capability.



Imbalanced Temperature:

- **Issue:** Data is heavily biased towards 25°C .
- **Impact:** Poor generalization for real-world varying temperature conditions.





Model Selection

Core Architecture: AE-BPNN :

- **Autoencoder (AE):** Unsupervised dimensionality reduction (compressing EIS to 20 key features).
- **BPNN:** Accurate SOH estimation using scaled conjugate gradient optimization.
- **Advantage:** Lightweight, low computational cost, and high R^2

Robustness Evaluation (Denoising) :

- **Method:** Noise Injection into raw EIS signals (Scenario B).
- **Objective:** To evaluate model stability against noisy data (simulating sensor noise).
- **Outcome:** Evaluates the model's denoising capability, aligning with the DAE (Denoising Autoencoder) concept.

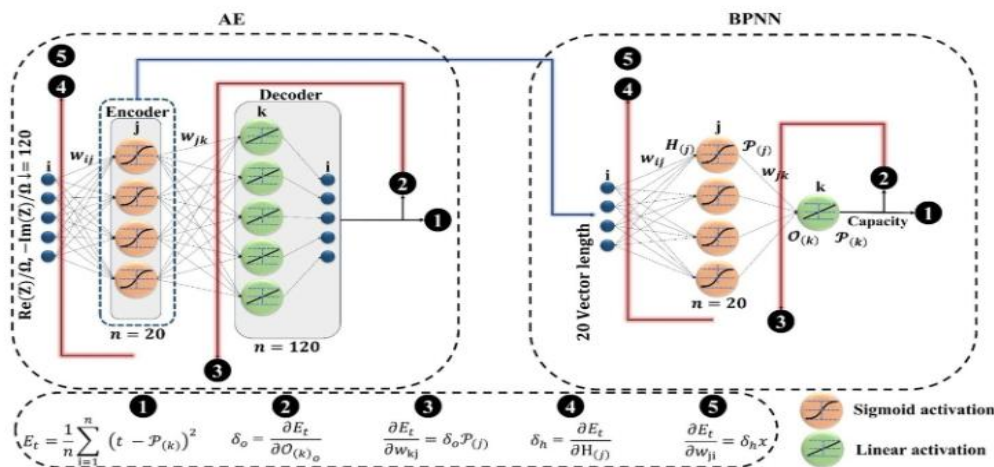


Fig. 1. Representation of the AE and BPNN architecture for SOH estimation.



Proposed Experiments

Pre-Experiment: Feature Enhancement

- **Upgrade:** Enhance EIS image resolution from 8×8 to 16×16 .
- **Goal:** Reduce information loss before model training.

Scenario A: Baseline Evaluation

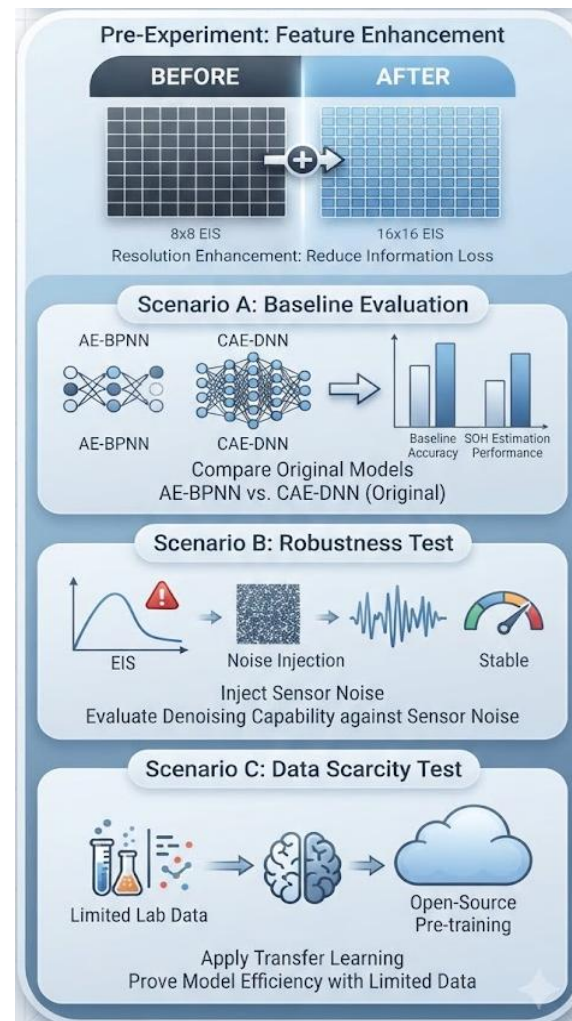
- **Test:** AE-BPNN vs. CAE-DNN (Original).
- **Metric:** Baseline accuracy and standard SOH estimation performance.

Scenario B: Robustness Test

- **Test:** Inject simulated noise into EIS signals.
- **Goal:** Evaluate stability and denoising capability against sensor noise.

Scenario C: Overcoming Data Scarcity

- **Test:** Apply Transfer Learning.
- **Goal:** Prove model efficiency using limited lab data combined with open-source pre-training.





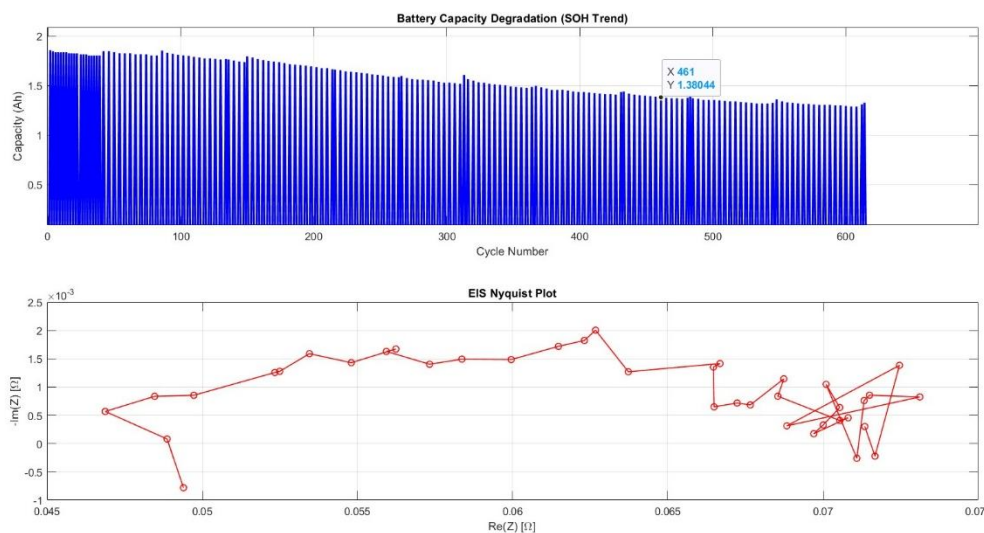
Open-Source Data Strategy

Public Dataset for Pre-Training :

- **Source:** NASA Prognostics Center of Excellence (PCoE).
- **Data Type:** Includes Impedance (EIS) measurements.
- **Features:** Provides both $\text{Re}(Z)$ and $-\text{Im}(Z)$ values matching our required format.

Implementation Plan :

- **Phase 1:** Pre-train AE-BPNN using extensive NASA data.
- **Phase 2:** Fine-tune the model using the 8 coin cells from our laboratory.
- **Advantage:** Immediate start on coding & model architecture without waiting for new lab experiments.



	Point	Re_Z	Neg_Im_Z
1	1	0.0713	3.0178e-04
2	2	0.0717	-2.2187e-04
3	3	0.0725	0.0014
4	4	0.0688	3.1369e-04
5	5	0.0731	8.2430e-04
6	6	0.0715	8.5541e-04
7	7	0.0713	7.5968e-04
8	8	0.0711	-2.5964e-04
9	9	0.0701	0.0010
10	10	0.0705	6.3676e-04
11	11	0.0700	3.2973e-04
12	12	0.0697	1.7479e-04
13	13	0.0708	4.5581e-04
14	14	0.0705	4.0800e-04
15	15	0.0685	8.3711e-04
16	16	0.0687	0.0011
17	17	0.0677	6.8346e-04
18	18	0.0672	7.1678e-04
19	19	0.0665	6.4951e-04
20	20	0.0665	0.0014
21	21	0.0667	0.0014

Figure : Data Extraction from MATLAB (BatteryAgingARC-FY08Q4 B0005)



Tentative Project Roadmap

Phase 1: Foundation & Data Prep



- **Data Cleaning:** NASA PCoE Dataset (Impedance values).



- **Pre-processing:** Image resolution enhancement (16×16 format).

Goal: Build a robust data pipeline for transfer learning.

Phase 2: Baseline & Robustness (Scenario A & B)



- **Scenario A:** AE-BPNN vs. CAE-DNN (Baseline accuracy test).
- **Scenario B:** Noise injection test to evaluate denoising capability.

Goal: Establish baseline performance and model stability.

Transfer Learning Application (Scenario C)



- **Scenario C:** Pre-Pre-training APNN using NASA data.



- Fine-tuning the model with limited laboratory (8 coin cells) data.

Goal: Demonstrate effective SOH estimation under Data Scarcity.

Phase 4: Analysis & Documentation



- Comparative analysis of results across all scenarios.



- Thesis writing & final result consolidation.

Goal: Finalize SOH estimation framework & research outcome.



Progress: Data Extraction & Consolidation

NASAImpedanceData			
Battery_ID	Cycle	Re_Z	Neg_Im_Z
Number	Number	Number	Number
Battery_ID	Cycle	Re_Z	Neg_Im_Z
1	41	0.07006937...	0.00047998...
1	41	0.06817886...	0.00119004...
1	41	0.06793257...	5.68268119...
1	41	0.06691839...	0.00087872...
1	41	0.06807105...	0.00019748...
1	41	0.06657097...	0.00058441...
1	41	0.06841817...	0.00095736...
1	41	0.06772414...	0.00047675...
1	41	0.06579331...	0.00064455...
1	41	0.06682466...	0.00080283...
1	41	0.06626554...	0.00088302...
1	41	0.06742609...	0.00122952...
1	41	0.06717687...	0.00156826...
1	41	0.06551759...	0.00114019...
1	41	0.06637541...	0.00099632...
1	41	0.06363403...	0.00163056...
1	41	0.06388080...	0.00073060...
1	41	0.06278200...	0.00126263...
1	41	0.06006457...	0.00121164...
1	41	0.06214646...	0.00163696...
1	41	0.06027082...	0.00174385...

```
clc; clear; close all;

% Define the target battery dataset names (NASA PCoE dataset)
battery_names = {'B0005', 'B0006', 'B0007', 'B0018'};
all_imp_data = []; % Initialize an empty array to store consolidated data

fprintf('Extracting EIS data from .mat files...\n');

% Loop through each battery dataset file
for b = 1:length(battery_names)
    batt_name = battery_names{b};

    % Check if the .mat file exists in the current directory before loading
    if isfile([batt_name, '.mat'])
        load([batt_name, '.mat']); % Load the .mat file into workspace
        data_struct = eval(batt_name); % Evaluate structure variable dynamically

        % Iterate through each cycle in the battery structure
        for i = 1:length(data_struct.cycle)
            % Check if the current cycle type is 'impedance' (EIS measurement)
            if strcmp(data_struct.cycle(i).type, 'impedance')
                imp_data = data_struct.cycle(i).data.Rectified_Impedance;

                % Use colon operator (:) to ensure data is strictly a Column Vector (Nx1)
                re = real(imp_data(:));
                neg_im = -imag(imp_data(:));

                % Create labels for Battery ID and Cycle Number as Column Vectors
                batt_id = repmat(b, length(re), 1);
                cycle_num = repmat(i, length(re), 1);
```

Automated Data Pipeline (MATLAB)

Source: NASA Prognostics Center of Excellence (PCoE).

- **Scope:** Extracted EIS data simultaneously from 4 baseline cells (B0005, B0006, B0007, B0018).
- **Features Extracted:** Battery_ID, Cycle, Re_Z, and Neg_Im_Z.
- B0005 = Battery_ID 1, B0006 = Battery_ID 2, B0007 = Battery_ID 3, B0018 = Battery_ID 4
- File Format .csv



Proof of Concept: Architecture Verification

```

10 class EIS_Autoencoder(nn.Module):
11     def __init__(self):
12         super(EIS_Autoencoder, self).__init__()
13
14         # 16x16 EIS image flattened into a 1D array = 256 input features
15         self.input_dim = 256
16
17         # The target number of compressed features (Latent space)
18         self.latent_dim = 20
19
20         # 1. Encoder Section: Compressing the 256 inputs down to 20 features
21         self.encoder = nn.Sequential(
22             nn.Linear(self.input_dim, 128),
23             nn.ReLU(),
24             nn.Linear(128, 64),
25             nn.ReLU(),
26             nn.Linear(64, self.latent_dim)
27         )
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

Initializing Data Pipeline and Model Architecture...

[SUCCESS] NASA Dataset loaded. Total records: 34593

[INFO] Testing Autoencoder forward pass with dummy 16x16 EIS data...

--- Model Architecture Verification ---

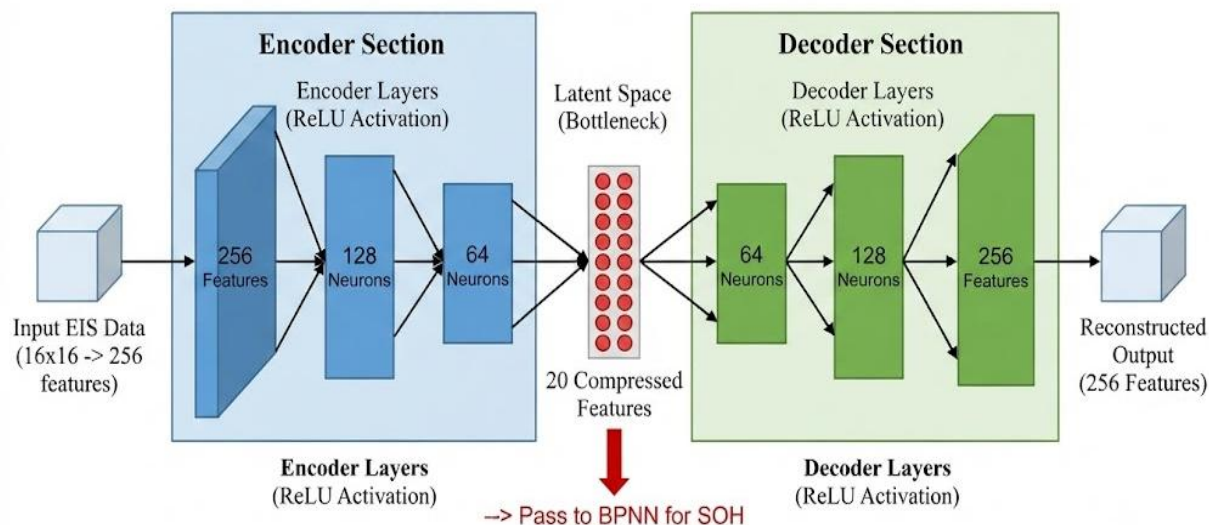
Input Shape: torch.Size([5, 256]) -> (Batch Size, Original Features)

Latent Feature Shape: torch.Size([5, 20]) -> (Batch Size, Compressed Features)

Output Shape: torch.Size([5, 256]) -> (Batch Size, Reconstructed Features)

Status: The Autoencoder blueprint is ready for data interpolation and training.

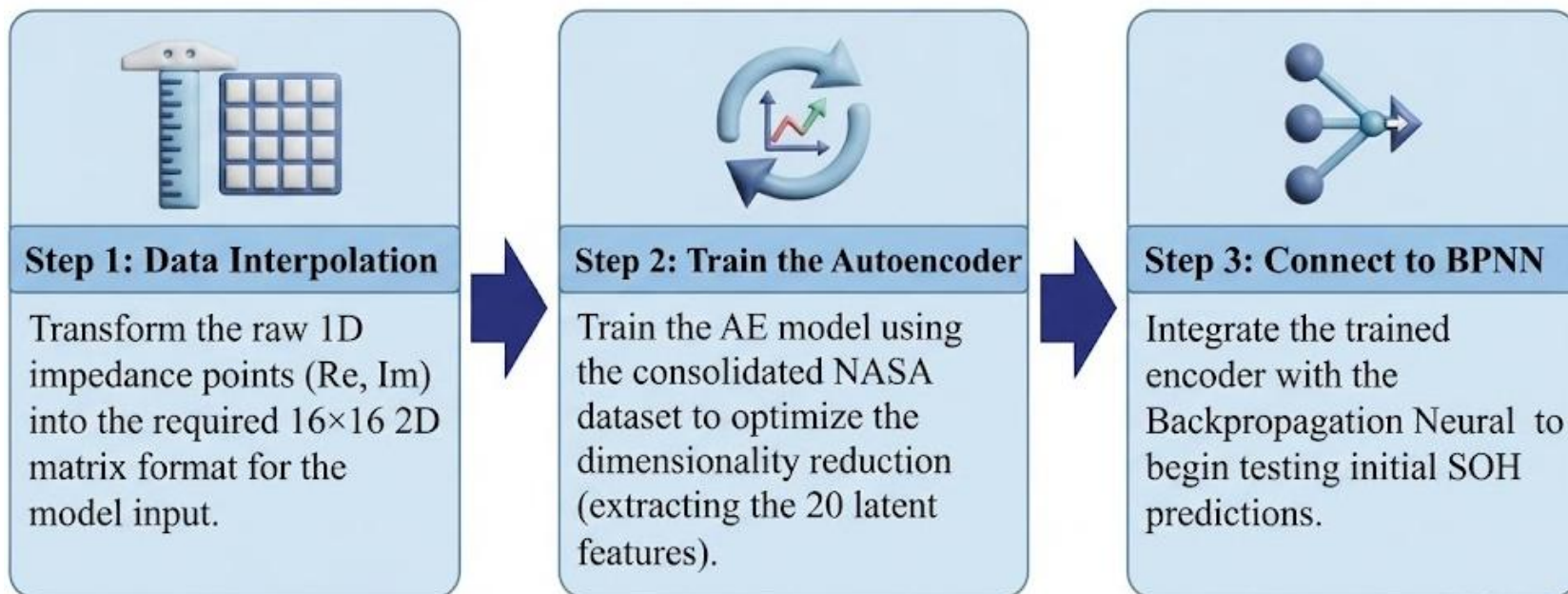
PS C:\Master Degree\Thesis\Autoencoder>





Immediate Next Steps

Phase 1: Foundation and Data Prep





Thank you